LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

STACK & QUEUE

Oleh:

Nor Khalisyah NIM.2510817120005

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack & Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Nor Khalisyah
NIM : 2510817120005

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIM. 2310817310009	NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN ii
DAFTAR ISI iii
DAFTAR GAMBAR iv
DAFTAR TABEL v
SOAL 1 1
A. Source Code 1
B. Pembahasan 2
SOAL 2 6
A. Source Code 7
B. Output Program 8
C. Pembahasan 8
SOAL 3 10
A. Source Code 10
B. Pembahasan 11
SOAL 4 15
A. Source Code 16
B. Output Program 17
C. Pembahasan 17
TAUTAN GIT 19

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban prob1.cpp 8
Gambar 2. Screenshot Hasil Jawaban prob2.cpp 17

DAFTAR TABEL

Table 1. Source Code stack.h 1
Table 2. Source Code stack.cpp 1
Table 3. Contoh input & output soal 2 7
Table 4. Source Code prob1.cpp 7
Table 5. Source Code queue.h 10
Table 6. Source Code queue.cpp 10
Table 7. Contoh input & output soal 4 16
Table 8. Souce Code prob2.cpp 16

SOAL 1

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 1. Source Code stack.h

```
#ifndef STACK_H
#define STACK_H

#define MAX 100

struct Stack {
    int data[MAX];
    int* top;
};

void init(Stack* s);
bool isEmpty(const Stack* s);
bool isFull(const Stack* s);
void push(Stack* s, int value);
void pop(Stack* s);
int peek(const Stack* s);

#endif
```

Table 2. Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s) {
    s->top = s->data - 1;
}

bool isEmpty(const Stack* s) {
    return s->top < s->data;
}

bool isFull(const Stack* s) {
```

```
13      return s->top == s->data + MAX - 1;
14  }
15
16  void push(Stack* s, int value) {
17       if (isFull(s)) throw std::overflow_error("Stack is
    full");
18      *(++s->top) = value;
19  }
20
21  void pop(Stack* s) {
22       if (isEmpty(s)) throw std::underflow_error("Stack is
    empty");
23      s->top--;
24  }
25
26  int peek(const Stack* s) {
27       if (isEmpty(s)) throw std::underflow_error("Stack is
    empty");
28      return *(s->top);
29  }
```

B. Pembahasan

Pembahasan [stack.h]

Pada baris [1], digunakan `#ifndef STACK_H` dan pada baris [2] digunakan `#define STACK_H` sebagai include guard untuk mencegah file header ini di-include lebih dari satu kali dalam proses kompilasi.

Pada baris [4], digunakan `#define MAX 100` untuk mendefinisikan konstanta `MAX` bernilai 100 yang digunakan sebagai ukuran maksimum array pada struct `Stack`.

Pada baris [6] – [9], terdapat definisi struct `Stack` yang memiliki dua anggota. Anggota pertama adalah `int data[MAX]` pada baris [7] yaitu array statis berukuran `MAX` yang berfungsi sebagai tempat penyimpanan elemen-elemen stack. Anggota kedua adalah `int* top` pada baris [8] yaitu pointer bertipe integer yang selalu menunjuk ke elemen paling atas stack.

Pada baris [11], dideklarasikan fungsi `init` yang menerima pointer ke `Stack` dan berfungsi untuk menginisialisasi stack ke keadaan kosong. Pada baris [12], dideklarasikan fungsi `isEmpty` yang menerima pointer konstan ke `Stack` dan mengembalikan nilai `bool` untuk mengecek apakah stack kosong. Pada baris [13], dideklarasikan fungsi

`isFull` yang menerima pointer konstan ke `Stack` dan mengembalikan nilai `bool` untuk mengecek apakah stack penuh. Pada baris [14], dideklarasikan fungsi `push` yang menerima pointer ke `Stack` dan nilai `int`, berfungsi untuk memasukkan elemen baru ke puncak stack.

Pada baris [15], dideklarasikan fungsi `pop` yang menerima pointer ke `Stack` dan berfungsi untuk menghapus elemen paling atas stack. Pada baris [16], dideklarasikan fungsi `peek` yang menerima pointer konstan ke `Stack` dan mengembalikan nilai `int`, berfungsi untuk melihat elemen paling atas stack tanpa menghapusnya. Pada baris [18], digunakan `#endif` sebagai penutup dari include guard.

Justifikasi kebenaran: Penggunaan `int* top` sebagai pointer memungkinkan pointer aritmetik `data - 1` sebagai sentinel kosong dan `data + MAX - 1` sebagai sentinel penuh. Saat kosong `top = data - 1` tidak pernah bisa diakses sebagai elemen valid. Saat ada 1 elemen `top = data` menunjuk elemen pertama. Saat penuh `top = data + MAX - 1` menunjuk elemen terakhir. Fungsi `isEmpty`, `isFull`, dan `peek` menggunakan `const Stack*` yang membuktikan fungsi-fungsi tersebut hanya membaca data tanpa mengubah isi stack sehingga tidak ada efek samping yang tidak diinginkan.

Pembahasan [stack.cpp]

Pada baris [1], digunakan `#include "stack.h"` untuk menghubungkan file implementasi dengan header `stack.h` sehingga struct `Stack` dan seluruh deklarasi fungsi dapat digunakan. Pada baris [2], digunakan `#include <stdexcept>` untuk mengaktifkan mekanisme exception handling seperti `std::overflow_error` dan `std::underflow_error`.

Pada baris [4] – [6], terdapat implementasi fungsi `init` yang menerima pointer ke `Stack`. Pada baris [5], fungsi mengatur `top` ke posisi `data - 1` yang berarti pointer berada satu posisi sebelum awal array sebagai penanda stack dalam keadaan kosong. Kebenaran algoritma ini terjamin karena indeks valid array dimulai dari 0 sehingga posisi `data - 1` tidak akan pernah bertabrakan dengan elemen valid. Kompleksitas waktu: O(1), kompleksitas ruang: O(1).

Pada baris [8] – [10], terdapat implementasi fungsi `isEmpty`. Pada baris [9], fungsi mengembalikan `true` jika `top < data` yaitu ketika pointer `top` masih berada sebelum awal array. Kebenaran algoritma ini terjamin karena `top` hanya akan berada di posisi tersebut ketika belum ada elemen yang di-push. Kompleksitas waktu: O(1), kompleksitas ruang: O(1).

Pada baris [12] – [14], terdapat implementasi fungsi `isFull`. Pada baris [13], fungsi mengembalikan `true` jika `top == data + MAX - 1` yaitu ketika pointer `top` sudah berada di posisi terakhir array. Kebenaran algoritma ini terjamin karena indeks maksimum array adalah `MAX - 1` sehingga kondisi ini hanya terpenuhi ketika seluruh slot array sudah terisi. Kompleksitas waktu: O(1), kompleksitas ruang: O(1).

Pada baris [16] – [19], terdapat implementasi fungsi `push` yang menerima pointer ke `Stack` dan nilai `value`. Pada baris [17], fungsi memeriksa apakah stack penuh menggunakan `isFull`, jika penuh melempar `std::overflow_error` sebagai penanganan kondisi tidak valid. Pada baris [18], pointer `top` dinaikkan satu posisi dengan pre-increment lalu nilai `value` disimpan di posisi tersebut. Kebenaran algoritma ini terjamin karena pre-increment memastikan `top` selalu menunjuk ke elemen teratas yang valid setelah operasi push. Kompleksitas waktu: O(1), kompleksitas ruang: O(1).

Pada baris [21] – [24], terdapat implementasi fungsi `pop` yang menerima pointer ke `Stack`. Pada baris [22], fungsi memeriksa apakah stack kosong menggunakan `isEmpty`, jika kosong melempar `std::underflow_error` sebagai penanganan kondisi tidak valid. Pada baris [23], pointer `top` diturunkan satu posisi. Kebenaran algoritma ini terjamin karena menurunkan `top` satu posisi secara efektif menghapus elemen teratas tanpa perlu menghapus datanya secara fisik karena elemen tersebut tidak akan bisa diakses lagi. Kompleksitas waktu: O(1), kompleksitas ruang: O(1).

Pada baris [26] – [29], terdapat implementasi fungsi `peek` yang menerima pointer konstan ke `Stack` dan mengembalikan nilai `int`. Pada baris [27], fungsi memeriksa apakah stack kosong menggunakan `isEmpty`, jika kosong melempar `std::underflow_error` sebagai penanganan kondisi tidak valid. Pada baris [28], fungsi mengembalikan nilai yang ditunjuk oleh pointer `top` tanpa mengubah posisinya. Kebenaran algoritma ini terjamin karena `top` selalu menunjuk ke elemen teratas yang

valid sehingga nilai yang dikembalikan pasti merupakan elemen paling atas stack. Kompleksitas waktu: O(1), kompleksitas ruang: O(1).

SOAL 2

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- Ai adalah nilai bilangan pada input
- Simbol yang valid hanya:
  - bilangan bulat
  - operator: +, -, *, /

Format Input

n

A1 A2 ... An

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Table 3. Contoh input & output soal 2

Input	**Output**
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Table 4. Source Code prob1.cpp

```
#include "stack.h"
#include <iostream>
#include <string>

int main() {
    int n;
    std::cin >> n;

    Stack s;
    init(&s);

    for (int i = 0; i < n; i++) {
        std::string token;
        std::cin >> token;

        if (token == "+" || token == "-" || token == "*" || token == "/") {
            int b = peek(&s); pop(&s);
            int a = peek(&s); pop(&s);
            int result;
            if (token == "+") result = a + b;
            else if (token == "-") result = a - b;
            else if (token == "*") result = a * b;
            else result = a / b;
```

```
24                 push(&s, result);
25             } else {
26                 push(&s, std::stoi(token));
27             }
28         }
29
30         std::cout << peek(&s) << std::endl;
31         return 0;
32     }
```

B. Output Program

Gambar 1. Screenshot Hasil Jawaban prob1.cpp

C. Pembahasan

Pada baris [1], digunakan `#include "stack.h"` untuk menghubungkan file ini dengan implementasi stack. Pada baris [2], digunakan `#include <iostream>` untuk operasi input output. Pada baris [3], digunakan `#include <string>` untuk menggunakan tipe data `std::string` dalam membaca token.

Pada baris [5] – [31], terdapat fungsi `main` sebagai titik masuk program. Pada baris [6], variabel `n` dideklarasikan untuk menyimpan jumlah simbol yang akan dibaca. Pada baris [7], nilai `n` dibaca dari input. Pada baris [9] – [10], dideklarasikan struct `Stack` bernama `s` lalu diinisialisasi menggunakan fungsi `init`.

Pada baris [12] – [27], terdapat perulangan sebanyak `n` kali untuk membaca setiap token. Pada baris [13] – [14], setiap token dibaca sebagai `string`. Pada baris [16],

dilakukan pengecekan apakah token adalah operator `+`, `-`, `*`, atau `/`. Jika ya, pada baris [17], diambil dua elemen teratas stack menggunakan `peek` dan `pop`, elemen pertama disimpan ke `b` dan elemen kedua ke `a`. Pada baris [19] – [23], dilakukan operasi aritmetika sesuai operator dan hasilnya disimpan ke `result`. Pada baris [24], hasil operasi di-push kembali ke stack. Jika token bukan operator, pada baris [26], token dikonversi ke integer menggunakan `std::stoi` lalu di-push ke stack.

Pada baris [30], nilai akhir yang tersisa di puncak stack diambil menggunakan `peek` dan dicetak sebagai hasil. Kebenaran algoritma ini terjamin karena notasi RPN (Reverse Polish Notation) memastikan bahwa ketika operator ditemukan, dua operand sebelumnya selalu sudah tersedia di stack.

Kompleksitas waktu: O(n) karena setiap token diproses tepat satu kali. Kompleksitas ruang: O(n) karena dalam kasus terburuk semua token adalah angka sehingga stack menyimpan n elemen.

SOAL 3

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 5. Source Code queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#define MAX 100

struct Queue {
    int data[MAX];
    int* front;
    int* rear;
};

void init(Queue* q);
bool isEmpty(const Queue* q);
bool isFull(const Queue* q);
void enqueue(Queue* q, int value);
void dequeue(Queue* q);
int front(const Queue* q);
int back(const Queue* q);

#endif
```

Table 6. Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q) {
    q->front = q->data;
    q->rear = q->data - 1;
}

bool isEmpty(const Queue* q) {
```

```
10      return q->rear < q->front;
11  }
12
13  bool isFull(const Queue* q) {
14      return q->rear == q->data + MAX - 1;
15  }
16
17  void enqueue(Queue* q, int value) {
18       if (isFull(q)) throw std::overflow_error("Queue is
    full");
19      *(++q->rear) = value;
20  }
21
22  void dequeue(Queue* q) {
23       if (isEmpty(q)) throw std::underflow_error("Queue is
    empty");
24      q->front++;
25  }
26
27  int front(const Queue* q) {
28       if (isEmpty(q)) throw std::underflow_error("Queue is
    empty");
29      return *(q->front);
30  }
31
32  int back(const Queue* q) {
33       if (isEmpty(q)) throw std::underflow_error("Queue is
    empty");
34      return *(q->rear);
35  }
```

B. Pembahasan

Pembahasan [queue.h]

Pada baris [1], digunakan `#ifndef QUEUE_H` dan pada baris [2] digunakan `#define QUEUE_H` sebagai include guard untuk mencegah file header ini di-include lebih dari satu kali dalam proses kompilasi.

Pada baris [4], digunakan `#define MAX 100` untuk mendefinisikan konstanta `MAX` bernilai 100 yang digunakan sebagai ukuran maksimum array pada struct `Queue`.

Pada baris [6] – [10], terdapat definisi struct `Queue` yang memiliki tiga anggota. Anggota pertama adalah `int data[MAX]` pada baris [7] yaitu array statis berukuran `MAX` yang berfungsi sebagai tempat penyimpanan elemen-elemen queue. Anggota kedua

adalah `int* front` pada baris [8] yaitu pointer yang selalu menunjuk ke elemen paling depan queue. Anggota ketiga adalah `int* rear` pada baris [9] yaitu pointer yang selalu menunjuk ke elemen paling belakang queue.

Pada baris [12], dideklarasikan fungsi `init` yang menerima pointer ke `Queue` dan berfungsi untuk menginisialisasi queue ke keadaan kosong. Pada baris [13], dideklarasikan fungsi `isEmpty` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `bool` untuk mengecek apakah queue kosong. Pada baris [14], dideklarasikan fungsi `isFull` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `bool` untuk mengecek apakah queue penuh. Pada baris [15], dideklarasikan fungsi `enqueue` yang menerima pointer ke `Queue` dan nilai `int`, berfungsi untuk memasukkan elemen baru ke belakang queue.

Pada baris [16], dideklarasikan fungsi `dequeue` yang menerima pointer ke `Queue` dan berfungsi untuk menghapus elemen paling depan queue. Pada baris [17], dideklarasikan fungsi `front` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `int` paling depan queue. Pada baris [18], dideklarasikan fungsi `back` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `int` paling belakang queue. Pada baris [20], digunakan `#endif` sebagai penutup dari include guard.

Justifikasi kebenaran: Penggunaan dua pointer `front` dan `rear` terbukti benar untuk merepresentasikan queue karena sesuai dengan prinsip FIFO (First In First Out). `front` selalu menunjuk elemen yang pertama masuk dan pertama keluar, sedangkan `rear` selalu menunjuk elemen yang terakhir masuk. Kondisi kosong ditandai dengan `rear < front` yang tidak mungkin terjadi dalam kondisi normal sehingga menjadi sentinel yang aman. Array statis berukuran 100 terbukti cukup sesuai batasan soal $n \leq 100$.

Pembahasan [queue.cpp]

Pada baris [1], digunakan `#include "queue.h"` untuk menghubungkan file implementasi dengan header `queue.h` sehingga struct `Queue` dan seluruh deklarasi fungsi dapat digunakan. Pada baris [2], digunakan `#include <stdexcept>` untuk mengaktifkan mekanisme exception handling seperti `std::overflow_error` dan `std::underflow_error`.

Pada baris [4] – [7], terdapat implementasi fungsi `init` yang menerima pointer ke `Queue`. Pada baris [5], pointer `front` diatur ke posisi awal array `data`. Pada baris [6], pointer `rear` diatur ke posisi `data - 1` sebagai penanda queue dalam keadaan kosong. Kebenaran algoritma ini terjamin karena dengan `rear < front` pada kondisi awal, fungsi `isEmpty` akan selalu mengembalikan `true` sebelum ada elemen yang dimasukkan. Kompleksitas waktu: O(1), kompleksitas ruang: O(1).

Pada baris [9] – [11], terdapat implementasi fungsi `isEmpty`. Pada baris [10], fungsi mengembalikan `true` jika `rear < front` yaitu ketika pointer `rear` berada sebelum pointer `front`. Kebenaran algoritma ini terjamin karena dalam kondisi normal `rear` selalu lebih besar atau sama dengan `front`, sehingga kondisi `rear < front` hanya terjadi saat queue kosong. Kompleksitas waktu: O(1), kompleksitas ruang: O(1).

Pada baris [13] – [15], terdapat implementasi fungsi `isFull`. Pada baris [14], fungsi mengembalikan `true` jika `rear == data + MAX - 1` yaitu ketika pointer `rear` sudah berada di posisi terakhir array. Kebenaran algoritma ini terjamin karena indeks maksimum array adalah `MAX - 1` sehingga kondisi ini hanya terpenuhi ketika seluruh slot array sudah terisi. Kompleksitas waktu: O(1), kompleksitas ruang: O(1).

Pada baris [17] – [20], terdapat implementasi fungsi `enqueue` yang menerima pointer ke `Queue` dan nilai `value`. Pada baris [18], fungsi memeriksa apakah queue penuh menggunakan `isFull`, jika penuh melempar `std::overflow_error` sebagai penanganan kondisi tidak valid. Pada baris [19], pointer `rear` dinaikkan satu posisi dengan pre-increment lalu nilai `value` disimpan di posisi tersebut. Kebenaran algoritma ini terjamin karena pre-increment memastikan `rear` selalu menunjuk ke elemen paling belakang yang valid setelah operasi enqueue. Kompleksitas waktu: O(1), kompleksitas ruang: O(1).

Pada baris [22] – [25], terdapat implementasi fungsi `dequeue` yang menerima pointer ke `Queue`. Pada baris [23], fungsi memeriksa apakah queue kosong menggunakan `isEmpty`, jika kosong melempar `std::underflow_error` sebagai penanganan kondisi tidak valid. Pada baris [24], pointer `front` dinaikkan satu posisi. Kebenaran algoritma ini terjamin karena menaikkan `front` satu posisi secara efektif menghapus

elemen terdepan tanpa perlu menghapus datanya secara fisik karena elemen tersebut tidak akan bisa diakses lagi. Kompleksitas waktu: O(1), kompleksitas ruang: O(1).

Pada baris [27] – [30], terdapat implementasi fungsi `front` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `int`. Pada baris [28], fungsi memeriksa apakah queue kosong menggunakan `isEmpty`, jika kosong melempar `std::underflow_error` sebagai penanganan kondisi tidak valid. Pada baris [29], fungsi mengembalikan nilai yang ditunjuk oleh pointer `front` tanpa mengubah posisinya. Kebenaran algoritma ini terjamin karena pointer `front` selalu menunjuk ke elemen terdepan yang valid. Kompleksitas waktu: O(1), kompleksitas ruang: O(1).

Pada baris [32] – [35], terdapat implementasi fungsi `back` yang menerima pointer konstan ke `Queue` dan mengembalikan nilai `int`. Pada baris [33], fungsi memeriksa apakah queue kosong menggunakan `isEmpty`, jika kosong melempar `std::underflow_error` sebagai penanganan kondisi tidak valid. Pada baris [34], fungsi mengembalikan nilai yang ditunjuk oleh pointer `rear` tanpa mengubah posisinya. Kebenaran algoritma ini terjamin karena pointer `rear` selalu menunjuk ke elemen paling belakang yang valid. Kompleksitas waktu: O(1), kompleksitas ruang: O(1).

SOAL 4

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- A_i: elemen array

Format Input

n

A1 A2 ... An

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Table 7. Contoh input & output soal 4

Input	**Output**
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Table 8. Souce Code prob2.cpp

```
#include "queue.h"
#include <iostream>

int main() {
    int n, k;
    std::cin >> n >> k;

    int arr[100];
    for (int i = 0; i < n; i++) std::cin >> arr[i];

    bool first = true;

    for (int i = 0; i <= n - k; i++) {
        Queue q;
        init(&q);
       for (int j = i; j < i + k; j++) enqueue(&q, arr[j]);

        int sum = 0;
        while (!isEmpty(&q)) {
            sum += front(&q);
            dequeue(&q);
        }

        if (!first) std::cout << " ";
        std::cout << sum;
        first = false;
    }

    std::cout << std::endl;
    return 0;
}
```

B. Output Program

Gambar 2. Screenshot Hasil Jawaban prob2.cpp

C. Pembahasan

Pada baris [1], digunakan `#include "queue.h"` untuk menghubungkan file ini dengan implementasi queue. Pada baris [2], digunakan `#include <iostream>` untuk operasi input output.

Pada baris [4] – [26], terdapat fungsi `main` sebagai titik masuk program. Pada baris [5], dideklarasikan variabel `n` untuk jumlah elemen dan `k` untuk ukuran window. Pada baris [6], nilai `n` dan `k` dibaca dari input.

Pada baris [8] – [9], dideklarasikan array `arr` berukuran 100 lalu diisi dengan `n` elemen dari input menggunakan perulangan. Pada baris [11], dideklarasikan variabel `first` bertipe `bool` untuk mengatur format output agar tidak ada spasi di awal.

Pada baris [13] – [24], terdapat perulangan utama sliding window yang berjalan dari `i = 0` hingga `i <= n - k`. Pada baris [14] – [15], dideklarasikan queue baru dan diinisialisasi setiap iterasi. Pada baris [16], sebanyak `k` elemen mulai dari indeks `i` hingga `i + k - 1` di-enqueue ke dalam queue. Pada baris [18] – [21], elemen queue dijumlahkan satu per satu menggunakan `front` dan `dequeue` hingga queue kosong. Pada baris [23] – [24], hasil penjumlahan dicetak dengan spasi sebagai pemisah antar window.

Pada baris [26], dicetak newline sebagai akhir output. Kebenaran algoritma ini terjamin karena setiap window selalu berisi tepat `k` elemen berurutan dan bergeser satu posisi ke kanan setiap iterasi hingga tidak ada lagi `k` elemen yang tersisa.

Kompleksitas waktu: $O(n \times k)$ karena untuk setiap posisi window dilakukan enqueue dan dequeue sebanyak `k` kali. Kompleksitas ruang: $O(k)$ karena queue hanya menyimpan `k` elemen setiap iterasi.

TAUTAN GIT

https://github.com/DSA26-ULM/module-2-stack-and-queue-norkhalisyah162007-crypto.git